

Module 15 - chapter 9

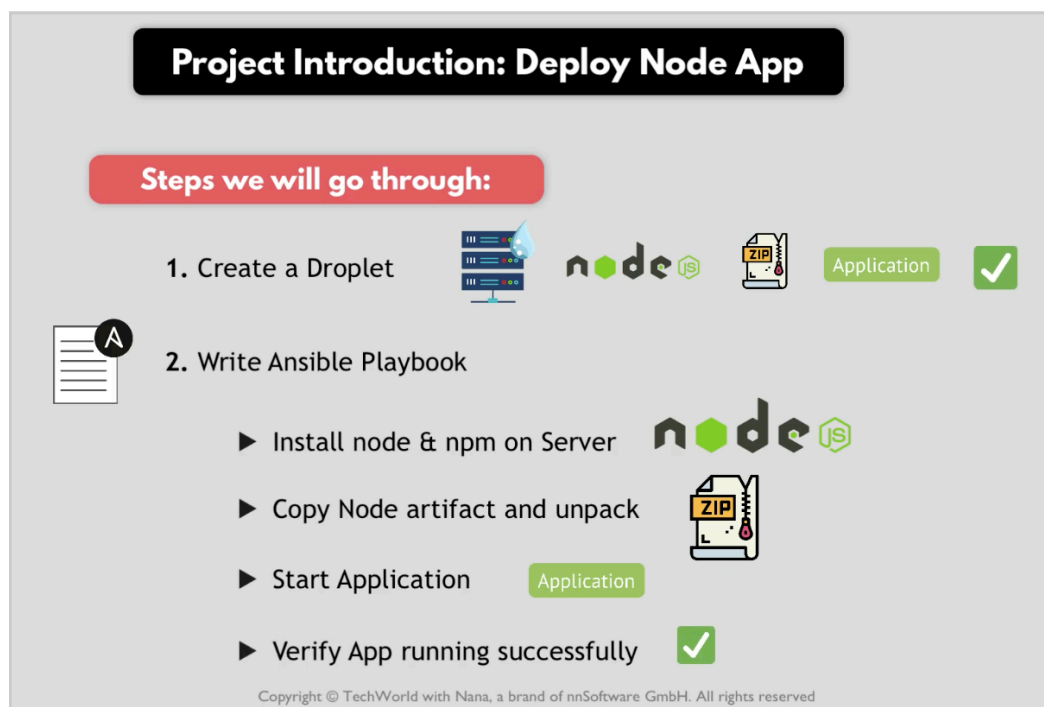
Project: Deploy Nodejs application - Part 1

The project used in this lecture can be found here:

<https://gitlab.com/twn-devops-bootcamp/latest/15-ansible/nodejs-app>

We are automating installing a nodes app in a server -> Digitalocean

1. Create a droplet
2. Write an Ansible playbook that:
 1. Installs node and npm both on the server
 2. Copy Node artifact to the server, unpack it there
 3. Start the application on the server using node command
 4. Check nodes app is running successfully



Preparation

Create server on Digital Ocean

by resource name or public IP (Cmd+B) Create ? 🔍 🔄

☒ Ubuntu RECOMMENDED 24.04 (LTS) x64 ▼

☐ Debian 13 x64

☐ Fedora Requires an SSH Key 44 x64 ▼

Show all OS images

Choose a Droplet Plan

Basic
Shared CPU

General Purpose
Dedicated CPU

CPU-Optimized
Dedicated CPU

More →

CPU Options

☒ Regular
Disk Type: SSD

☐ Premium AMD
Disk Type: NVMe SSD

☐ Premium Intel
Disk Type: NVMe SSD

Select a Plan

☐ \$4.00/mo
1 vCPU - 512 MB RAM - 10 GB SSD - 500 GB Transfer

☒ \$6.00/mo
1 vCPU - 1 GB RAM - 25 GB SSD - 1000 GB Transfer

☐ \$12.00/mo
1 vCPU - 2 GB RAM - 50 GB SSD - 2 TB Transfer

Summary

Compute \$6.00/mo
Plan Type: Basic
CPU Option: Regular
vCPU: 1
RAM: 1 GB
Disk: 25 GB
Bandwidth: 1000 GB
Slug: s-1vcpu-1gb

Total cost \$6.00/month
\$0.009/hour

Create Droplet

Select an SSH Key to create this Droplet

Want to maximize efficiency and cost?
Programmatically manage Droplets in a repeatable and re-usable way with our API. Create via API

Authentication

SSH Keys

Password

Add an SSH Key for authentication

An SSH key pair is a more secure way to connect to your Droplet

Add SSH Key

☒ Select All 1/1 ▲

☒ my-droplet-key

Create Droplet

Want to maximize efficiency and cost?
Programmatically manage Droplets in a repeatable and re-usable way with our API. Create via API

Quantity

Deploy multiple Droplets with the same configuration.

— +

Give your Droplet a name

Can only contain alphanumeric characters, dashes, and periods only.

Finalize

Select a project

SSH-Demo ▼

Tags

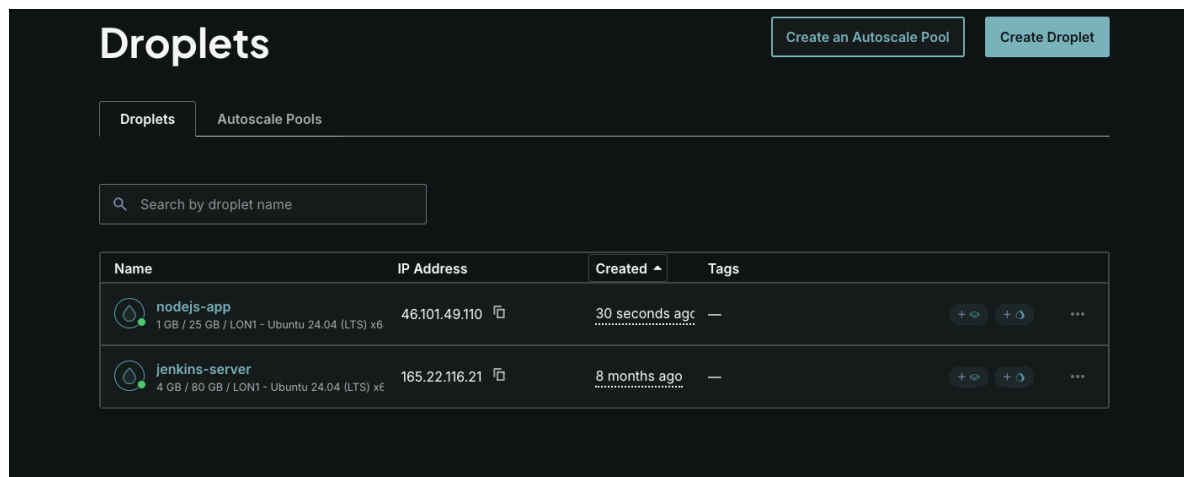
May contain letters, numbers, colons, dashes, and underscores.

Total cost \$6.00/month
\$0.009/hour

Create Droplet

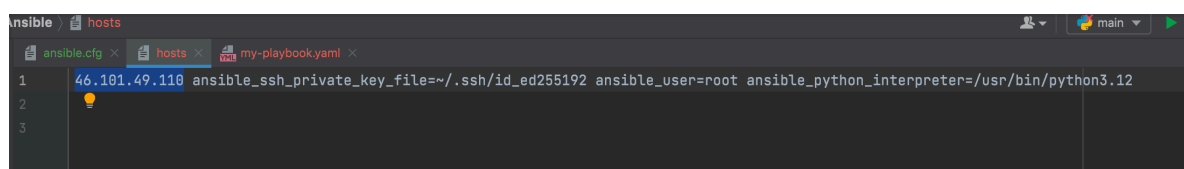
Want to maximize efficiency and cost?
Programmatically manage Droplets in a repeatable and re-usable way with our API. Create via API

And droplet is created:



Write Playbook

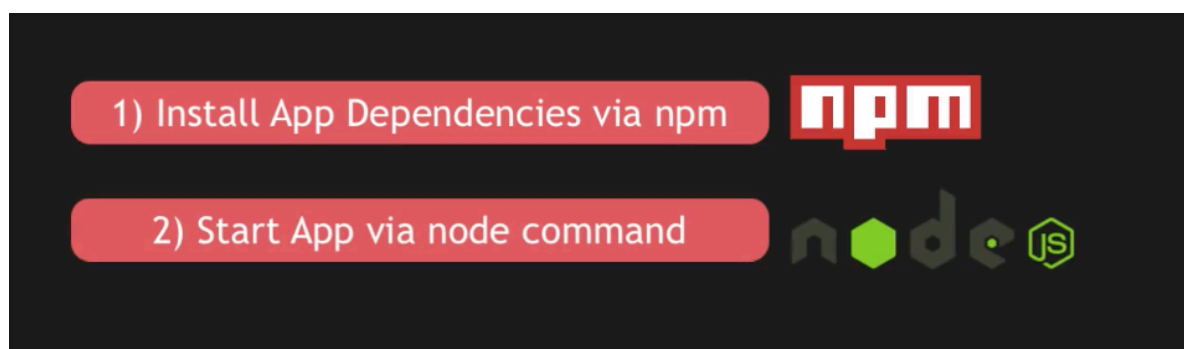
In the inventory file 'hosts' I copy the IP address of the new droplet along with the



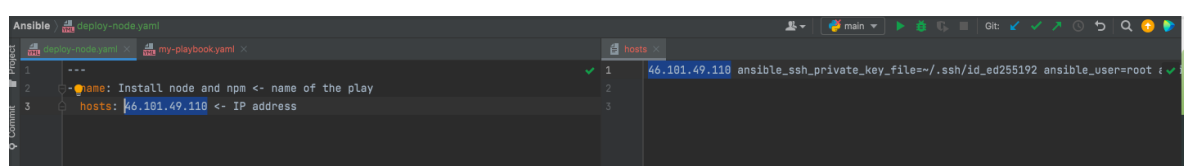
Install node and npm

Create a new playbook called 'deploy-node.yaml'

And install npm and node so we later can:

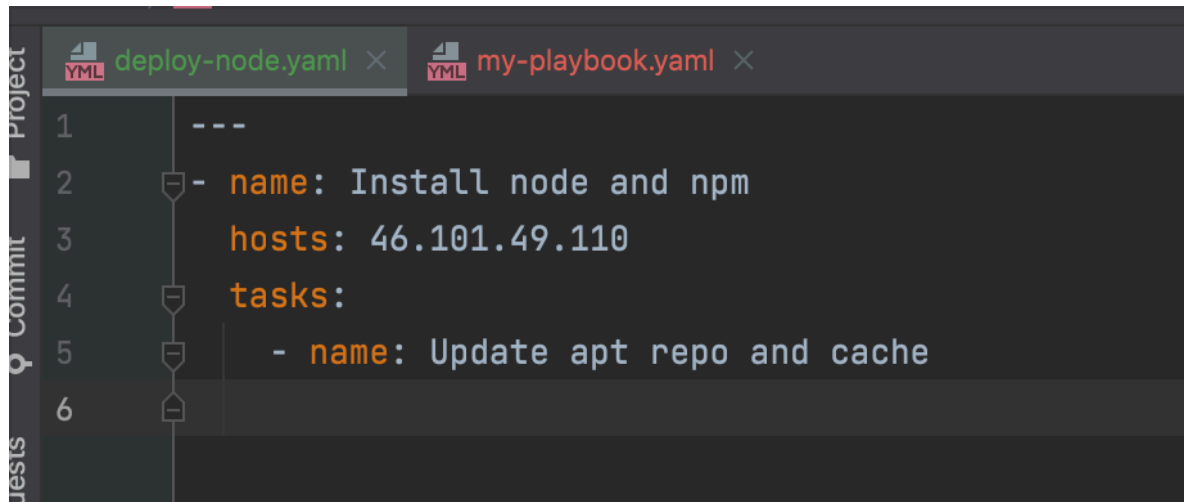


So we named the play and add the IP of the host:



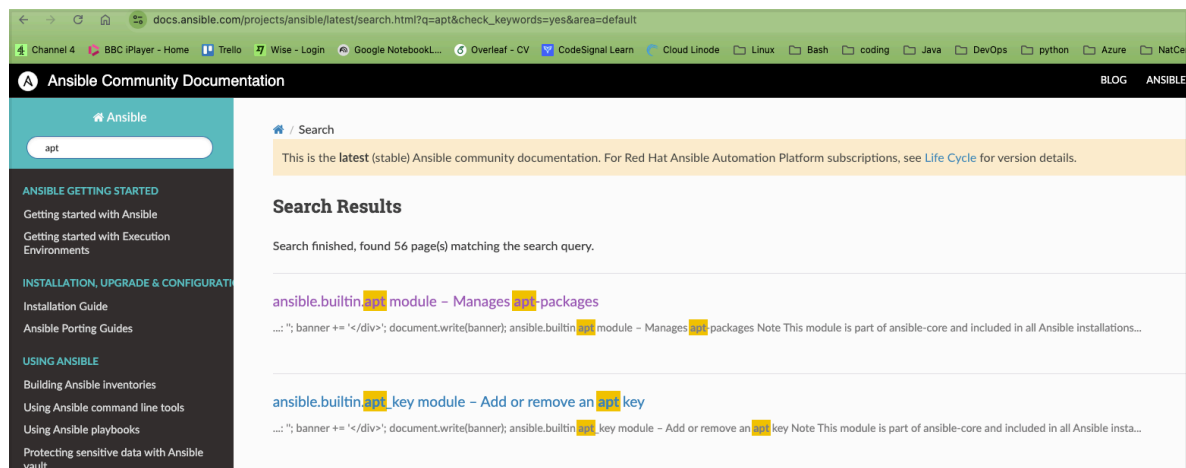
Then we add the tasks, and the first task is to update apt so we write a name to

describe the task:



```
---
- name: Install node and npm
  hosts: 46.101.49.110
  tasks:
    - name: Update apt repo and cache
```

And we can use the 'apt' module and we can see that apt is part of ansible.builtin module



docs.ansible.com/projects/ansible/latest/collections/ansible/builtin/index.html

Channel 4 BBC iPlayer - Home Trello Wise - Login Google NotebookL... Overleaf - CV CodeSignal Learn Cloud Linode Linux Bash coding Java De

Ansible Community Documentation

- Ansible.Mysql
- Ansible.Netcommon
- Ansible.Posix
- Ansible.Utils
- Ansible.Windows

- Collections in the Arista Namespace
- Collections in the Awx Namespace
- Collections in the Azure Namespace
- Collections in the Check_point Namespace
- Collections in the Chocolatey Namespace
- Collections in the Cisco Namespace
- Collections in the Cloud Namespace
- Collections in the Cloudscale_ch Namespace
- Collections in the Community Namespace
- Collections in the Containers Namespace
- Collections in the Cyberark Namespace
- Collections in the Dellemc Namespace
- Collections in the F5networks Namespace
- Collections in the Fortinet Namespace
- Collections in the Frr Namespace
- Collections in the Gluster Namespace
- Collections in the Google Namespace
- Collections in the Grafana Namespace
- Collections in the Graphiant Namespace
- Collections in the Hetzner Namespace
- Collections in the Hitachivantara

Ansible.Builtin

Collection version 2.21.2.post0

- Description
- Communication
- Plugin Index

Description

These are all modules and plugins contained in ansible-core.

Author:

- Ansible, Inc.

[Issue Tracker](#) [Repository \(Sources\)](#)

Communication

- Forum: [Ansible Forum](#).
- Matrix room: [#users:ansible.im](#) : General usage and support questions.
- IRC channel: [#ansible](#) (Libera network): [General usage and support questions](#).

Plugin Index

These are the plugins in the ansible.builtin collection:

Modules

- [add_host module](#) – Add a host (and alternatively a group) to the ansible-playbook in-memory inventory
- [apt module](#) – Manages apt-packages
- [apt_key module](#) – Add or remove an apt key **DEPRECATED**
- [apt_repository module](#) – Add and remove APT repositories **DEPRECATED**

So we can call it just by using 'apt' no need for the full name.

And we can see in the documentation that there is 'update cache'

https://docs.ansible.com/projects/ansible/latest/collections/ansible/builtin/apt_module.html#ansible-collections-ansible-builtin-apt-module

update_cache aliases: update-cache boolean	Run the equivalent of <code>apt-get update</code> before the operation. Can be run as part of the package installation or as a separate step. Default is not to update the cache. Choices: - false - true

I can do it using equal like below:

```

1  ---
2  - name: Install node and npm
3    hosts: 46.101.49.110
4    tasks:
5      - name: Update apt repo and cache
6        apt: update_cache=yes

```

Or I can do it:

```

1  ---
2  - name: Install node and npm
3    hosts: 46.101.49.110
4    tasks:
5      - name: Update apt repo and cache
6        apt:
7          update_cache: yes

```

Now we will add to force apt get:

	• <code>true</code>
<code>force_apt_get</code> boolean	Force usage of apt-get instead of aptitude. Choices: • <code>false</code> ← (default) • <code>true</code>
<code>install_recommends</code>	Corresponds to the <code>--no-install-recommends</code> option for <code>apt</code> . <code>true</code> installs recommended

```
Ansible > deploy-node.yaml
1 ---
2 - name: Install node and npm
3   hosts: 46.101.49.110
4   tasks:
5     - name: Update apt repo and cache
6       apt: update_cache=yes force_apt_get=yes
```

And then we add cache valid time which is in seconds and we will set it to 3600

<code>cache_valid_time</code> integer	Update the apt cache if it is older than the <code>cache_valid_time</code> . This option is set in seconds. As of Ansible 2.4, if explicitly set, this sets <code>update_cache=yes</code> . Default: 0
--	--

So it is not going to update cache to another 3600 seconds.

```
1 ---
2 - name: Install node and npm
3   hosts: 46.101.49.110
4   tasks:
5     - name: Update apt repo and cache
6       apt: update_cache=yes force_apt_get=yes cache_valid_time=3600
```

And line 6 above is the equivalent of 'apt update' command 

- `update_cache: yes` — this is what actually triggers a cache refresh, functionally equivalent to apt-get update. Without it, the apt module won't touch the package index at all.
- `cache_valid_time: 3600` — makes the refresh conditional. Ansible checks the timestamp of the last successful update (via `/var/lib/apt/periodic/update-success-stamp`, or the cache dir mtime as fallback). If it's newer than 3600 seconds old, the update is skipped entirely. So this is not "run apt update" every time — it's "run apt update, but only if the cache is stale (>1 hour old)." That's the key difference from a plain apt update, which always hits the network.
- `force_apt_get: yes` — controls how the module does its work under the hood. By default the apt module prefers the python-apt library bindings (and will use aptitude for some operations if it's installed) rather than shelling out to the apt-

get binary. Setting this forces it to always invoke the actual apt-get command-line tool instead, for more predictable/consistent behavior matching what you'd get running apt-get by hand.

So put together, that combination means: "refresh the package index by shelling out to apt-get update, but skip it if we already did so within the last hour." It's a throttled (slows something down), idempotent version of apt update — optimized for repeated playbook runs where you don't want every run to hit the network and re-download indexes if they were just refreshed.

Refresh the package index — apt doesn't know what packages/versions are available just by looking at your system; it keeps a local cache of that information (lists of package names, versions, dependencies, and which repo they come from) in /var/lib/apt/lists/. That cache is built by downloading index files from each repository URL in your sources list. "Refreshing the index" means re-downloading those files so your local cache matches what's currently in the remote repos — that's literally what apt update does. It doesn't install or upgrade anything; it just updates apt's knowledge of what's available. If you skip it, apt install somepkg might fail (package not found) or install an outdated version, because apt is working off a stale local copy.

"throttle" generally means limit the rate of something happening — not necessarily making the thing itself slower once it runs.

In networking, "throttling bandwidth" does mean slowing down a transfer (e.g., capping it at 1MB/s). But in this Ansible case, the individual apt update operation isn't made slower — it runs at full speed when it runs. What's being limited is how often it's allowed to happen: at most once per hour, per the cache_valid_time window. Outside that window, it's skipped completely (zero cost) rather than executed slowly.

So a more accurate paraphrase here would be "rate-limited" or "skipped if done recently," rather than "slowed down." The general sense of "throttle = limit the rate" still applies, just via skipping rather than slowing.

NEXT

Now we will install nodes and npm which depends on which server we are using, which linux distro we are on. Check the installation guide. our droplet is on ubuntu which already has npm and nodes in the repository so we don't have to add it to the repository, we just need to install it.

— little tangent on apt and repos —

The "repo" isn't baked into apt, it's a set of external repository URLs listed in /etc/apt/sources.list (and /etc/apt/sources.list.d/). Ubuntu ships with several of these enabled by default:

- main – Canonical-supported, open source

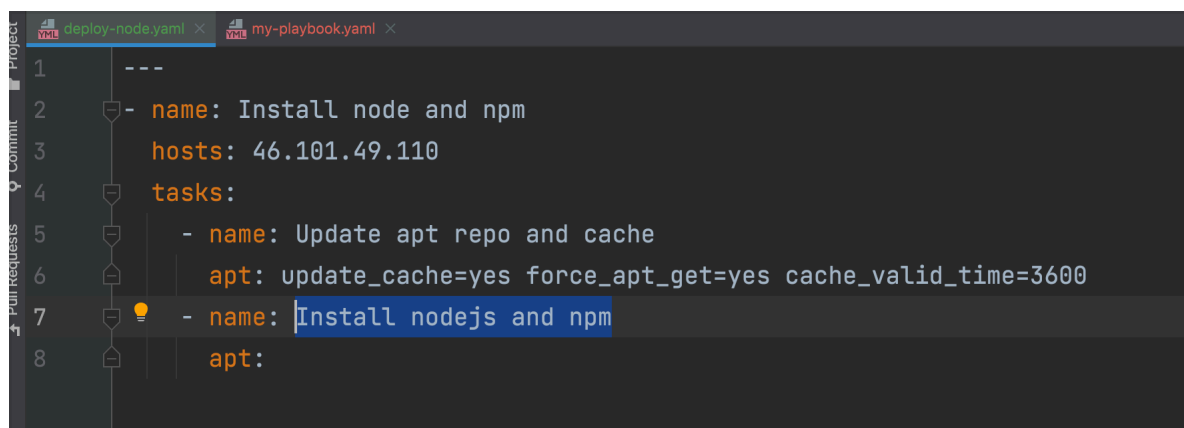
- restricted – Canonical-supported, proprietary drivers/firmware
- universe – community-maintained, open source
- multiverse – community-maintained, restricted licensing

If a package isn't in any enabled repo, you add a new one (a PPA, a vendor's official repo, etc.) along with its GPG signing key, then apt update before installing.

Your npm example is actually a case where this gets misunderstood: npm/nodejs is in Ubuntu's default universe repo, so `sudo apt install nodejs npm` works out of the box — no extra repo needed. The reason people add a third-party repo (e.g. NodeSource) isn't that it's missing, it's that Ubuntu's default version is often quite outdated. Adding the NodeSource repo gets you current Node/npm releases instead of whatever's frozen in that Ubuntu release's archive.

So the pattern "not in repo → add a repo" is correct, but a more common real-world trigger is "in repo, but too old → add a repo with newer versions." Other ways around this besides adding a repo: snap install, flatpak, or a version manager like nvm for Node specifically.

—end tangent —



```
1 ---
2 - name: Install node and npm
3   hosts: 46.101.49.110
4   tasks:
5     - name: Update apt repo and cache
6       apt: update_cache=yes force_apt_get=yes cache_valid_time=3600
7     - name: Install nodejs and npm
8       apt:
```

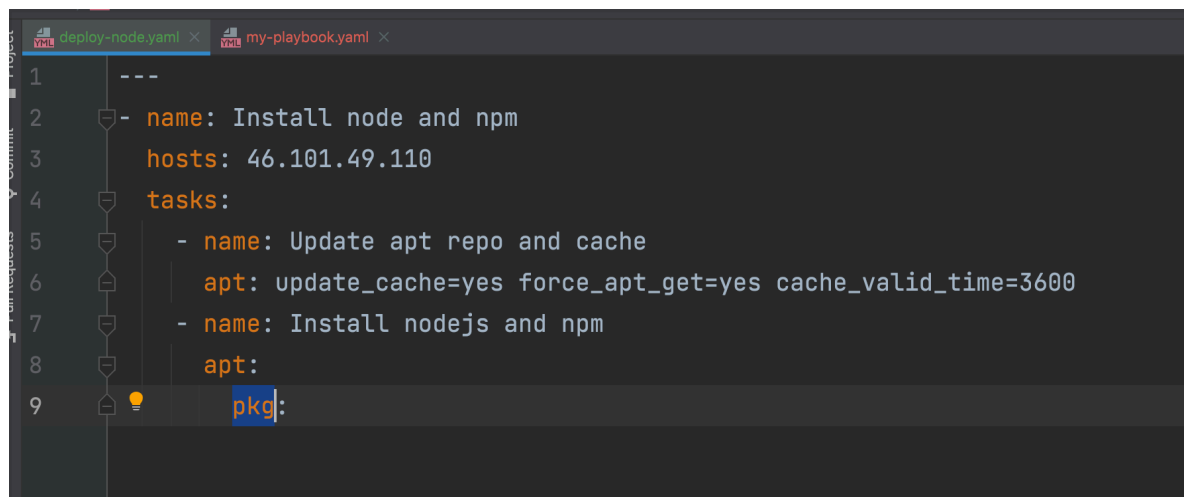
If I want to install only one package I will do

```
1 ---
2 - name: Install node and npm
3   hosts: 46.101.49.110
4   tasks:
5     - name: Update apt repo and cache
6       apt: update_cache=yes force_apt_get=yes cache_valid_time=3600
7     - name: Install nodejs and npm
8       apt: name=nodejs
```

And add the state like we did here:

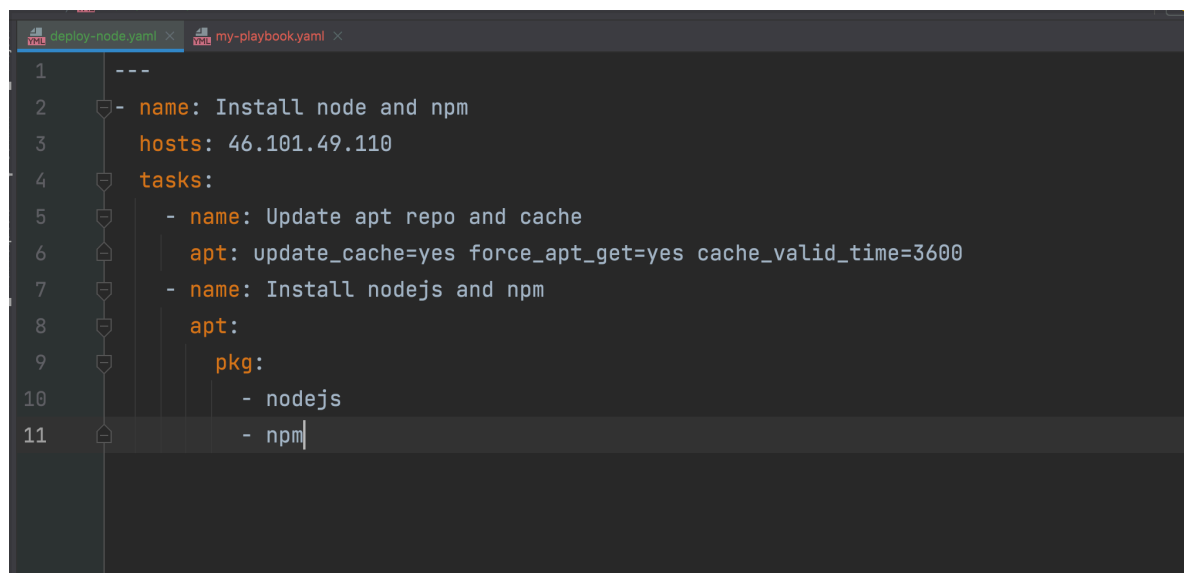
```
1 ---
2 - name: Configure nginx web server
3   hosts: webserver
4   tasks:
5     - name: uninstall nginx server
6       apt:
7         name: nginx=1.24.*
8         state: absent
9     - name: stop nginx server
10      service:
11        name: nginx
12        state: stopped
```

But we are installing more than 1, we are installing multiple packages so we use the attribute 'pkg' as in packages



```
1 ---
2 - name: Install node and npm
3   hosts: 46.101.49.110
4   tasks:
5     - name: Update apt repo and cache
6       apt: update_cache=yes force_apt_get=yes cache_valid_time=3600
7     - name: Install nodejs and npm
8       apt:
9         pkg:
```

And then we add the list:



```
1 ---
2 - name: Install node and npm
3   hosts: 46.101.49.110
4   tasks:
5     - name: Update apt repo and cache
6       apt: update_cache=yes force_apt_get=yes cache_valid_time=3600
7     - name: Install nodejs and npm
8       apt:
9         pkg:
10           - nodejs
11           - npm
```

And we can see the documentation showing an example:

Examples

```
- name: Install apache httpd (state=present is optional)
  ansible.builtin apt:
    name: apache2
    state: present

- name: Update repositories cache and install "foo" package
  ansible.builtin apt:
    name: foo
    update_cache: yes

- name: Remove "foo" package
  ansible.builtin apt:
    name: foo
    state: absent

- name: Install the package "foo"
  ansible.builtin apt:
    name: foo

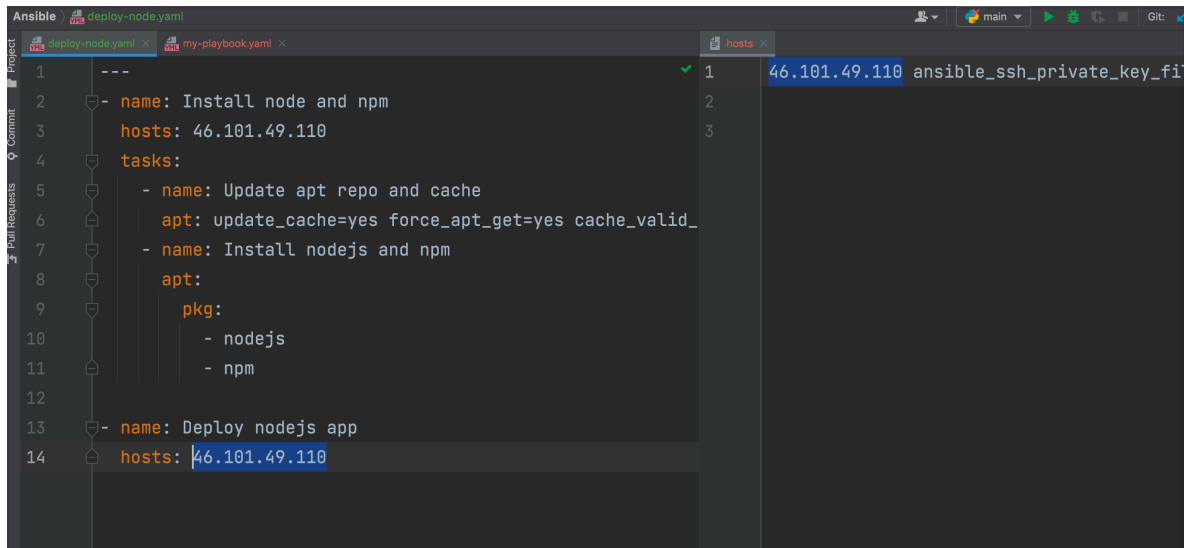
- name: Install a list of packages
  ansible.builtin apt:
    pkg:
      - foo
      - foo-tools

- name: Install the version '1.00' of package "foo"
  ansible.builtin apt:
    name: foo=1.00

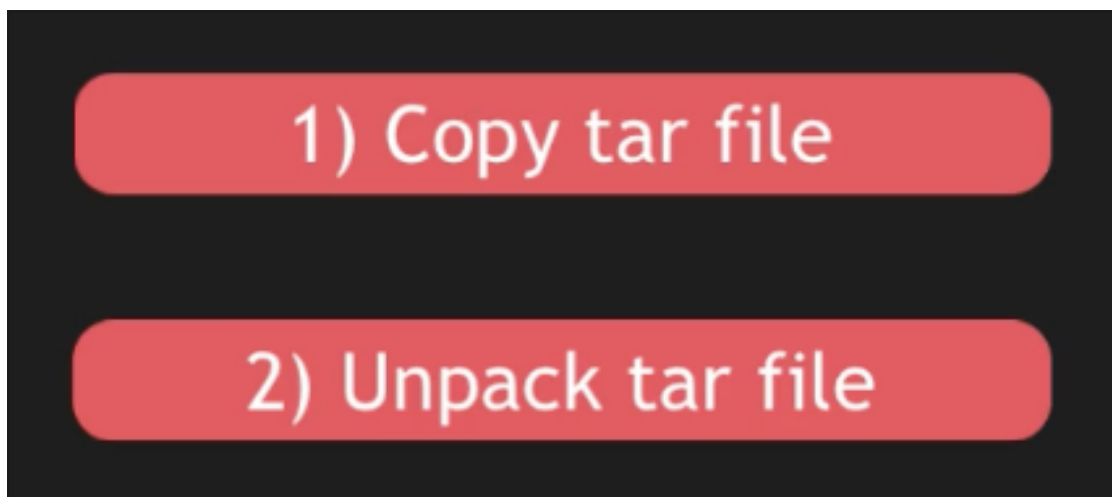
- name: Update the repository cache and update package "nginx" to latest version
  ansible.builtin apt:
    name: nginx
```

Copy and unpack the tar file

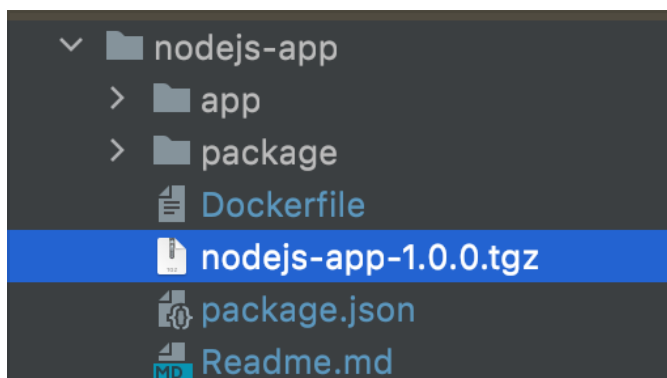
Let's create another play in the 'deploy-node.yaml' playbook file.



And now we want to add tasks to the new play:



This is the tar file 'nodes-app-1.00.tgz'

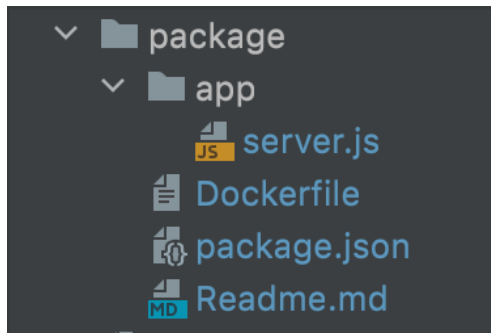


Which was created by 'npm pack' command.

And when the tar file is unpacked we get a folder called 'package' that contains:

```
% ls package
Dockerfile      Readme.md      app            package.json
```

So we will copy the tar on the server -> then we will unpack it and we will end up with a 'package' folder with the above contents, also see below:



So to copy the tar file we need its path or location:

```
> pwd
/Users/astrid/PythonProject/Ansible
> ls
ansible.cfg  deploy-node.yaml hosts  my-playbook.yaml nodejs-app
> cd nodejs-app
> ls
Dockerfile  Readme.md  app  nodejs-app-1.0.0.tgz package  package.json
> cd ..
> pwd
/Users/astrid/PythonProject/Ansible
> cd nodejs-app
> pwd
/Users/astrid/PythonProject/Ansible/nodejs-app
```

We will use the module 'copy'

```
13  - name: Deploy nodejs app
14      hosts: 46.101.49.110
15      tasks:
16      - name: Copy nodejs folder to a server
17          copy:
```

And add the source 'src' and the destination 'dest'

```

13  - name: Deploy nodejs app
14    hosts: 46.101.49.110
15    tasks:
16      - name: Copy nodejs folder to a server
17        copy:
18          src: /Users/astrid/PythonProject/Ansible/nodejs-app/nodejs-app-1.0.0.tgz
19          dest:

```

When we ssh into the server we use the user 'root'

```

hosts
46.101.49.110 ansible_ssh_private_key_file=~/.ssh/id_ed255192 ansible_user=root ansible_pythor

```

So when we copy something into the server it will go to the root users home directory which is /root

```

17    copy:
18      src: /Users/astrid/PythonProject/Ansible/nodejs-app/nodejs-app-1.0.0.tgz
19      dest: /root/

```

We want the tar to have a new name 'app-1.0.0.tgz'

```

16    - name: Copy nodejs folder to a server
17      copy:
18        src: /Users/astrid/PythonProject/Ansible/nodejs-app/nodejs-app-1.0.0.tgz
19        dest: /root/app-1.0.0.tgz

```

Next, after copying the tar file into the server we will unpack it using module 'unarchive'

```

21    unarchive:
22      src: /root/app-1.0.0.tgz
23      dest: /root/

```

And it will create the 'package' folder in the destination folder -> root/

And we need another attribute:

```

20     - name: Unpack the nodejs file
21       unarchive:
22         src: /root/app-1.0.0.tgz
23         dest: /root/
24         remote_src: yes

```

To indicate that the source path in line 22 is on the **remote server** otherwise it will look for the path in line 22 in our local machine.

Now let's run what we have so far

-> `ansible-playbook -i hosts deploy-node.yaml`

```

> ansible-playbook -i hosts deploy-node.yaml

PLAY [Install node and npm] *****

TASK [Gathering Facts] *****
ok: [46.101.49.110]

TASK [Update apt repo and cache] *****
changed: [46.101.49.110]

TASK [Install nodejs and npm] *****
changed: [46.101.49.110]

PLAY [Deploy nodejs app] *****

TASK [Gathering Facts] *****
ok: [46.101.49.110]

```



```

TASK [Copy nodejs folder to a server] *****
changed: [46.101.49.110]

TASK [Unpack the nodejs file] *****
changed: [46.101.49.110]

PLAY RECAP *****
46.101.49.110      : ok=6    changed=4    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

~/Pyt/Ansible ▶ Module_15/Ansible_chapter-9 +20 13 75 ▶ 3m 32s 17:37:56

```

Now let's ssh into the remote server to check

-> ssh root@46.101.49.110

```

Last login: Tue Aug  4 16:37:56 2026 from 140.228.47.252
root@nodejs-app:~# pwd
/root
root@nodejs-app:~# ls
app-1.0.0.tgz  package
root@nodejs-app:~#

```

And the tar file is there and also it has been unpacked hence we can see the folder 'package'

And inside the 'package' folder can see the files that we expect to have;

```

root@nodejs-app:~# ls package/
Dockerfile  Readme.md  app  package.json
root@nodejs-app:~#

```

We can refactor the yaml file:

```

14 hosts: 46.101.49.110
15 tasks:
16   - name: Copy nodejs folder to a server
17     copy:
18       src: /Users/astrid/PythonProject/Ansible/nodejs-app/nodejs-app-1.0.0.tgz
19       dest: /root/app-1.0.0.tgz
20   - name: Unpack the nodejs file
21     unarchive:
22       src: /root/app-1.0.0.tgz
23       dest: /root/
24       remote_src: yes

```

If unarchive can look into our local machine when we don't use `remote_src: yes` then we can just do 'unarchive' module and no need for 'copy' module:

```

15 tasks:
16   - name: Unpack the nodejs file
17     unarchive:
18       src: /Users/astrid/PythonProject/Ansible/nodejs-app/nodejs-app-1.0.0.tgz
19       dest: /root/
20

```

And I remove the files from the server:

```

X root@nodejs-app: ~ (ssh)
root@nodejs-app:~# rm app-1.0.0.tgz
root@nodejs-app:~# ls
package
root@nodejs-app:~# rm package/
rm: cannot remove 'package/': Is a directory
root@nodejs-app:~# ls
package
root@nodejs-app:~# rm -rf package/
root@nodejs-app:~# ls
root@nodejs-app:~#

```

And execute the playbook again:

```

) ansible-playbook -i hosts deploy-node.yaml

PLAY [Install node and npm] *****

TASK [Gathering Facts] *****
ok: [46.101.49.110]

TASK [Update apt repo and cache] *****
ok: [46.101.49.110]

TASK [Install nodejs and npm] *****
ok: [46.101.49.110]

PLAY [Deploy nodejs app] *****

TASK [Gathering Facts] *****
ok: [46.101.49.110]

TASK [Unpack the nodejs file] *****
changed: [46.101.49.110]

PLAY RECAP *****
46.101.49.110 : ok=5    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

```

Most of the tasks above are green -> no changes (apt update didn't get executed because we try to run it within 3600 seconds so the cache is still valid and there is not need to update). Only the unpacking task.

I go back to the remote server

-> ls

```

root@nodejs-app:~# ls
package
root@nodejs-app:~# █

```

So we don't have the tar file as we didn't need to copy it into the server as we can access the local machine and the unpack it directly into the server, hence we only have the 'package' file.